Day 4

Add AI to the Catalog,

then test the AI

6 hours · 3 modules · 3 labs · the day Day 3 was secretly setting up

Where we left off

Days 1-3: a tested, CI-green catalog with a typed Python client.

Today: **add an LLM agent — and lock it under the same testing discipline.**

```
Day 3      Day 4 (today)
─────      ─────────────

client → CatalogAgent(api_client, llm_client)
            ├── tools = APIClient methods
            ├── plan / act / observe loop
            └── pytest suite: tools + schema + loop + golden evals
```

Catch-up: `cp -r ../checkpoints/day-4-start/. .`

Today's arc

Module	~40 min	80 min lab
1	LLMs + structured outputs	Lab 10: NL query → catalog filter
2	Tools as functions + agent loop	Lab 11: Build the `CatalogAgent`
3	**Testing AI tools and outputs** ⭐	Lab 12: Test the agent

End-of-day: an agentic, **tested** Python system on your laptop.

Module 1

LLM Fundamentals + Structured Outputs

~40 min · then 80 min lab

AI → ML → GenAI → Agentic

```
AI           any computational system that "appears intelligent"
ML           systems that learn patterns from data
GenAI        ML that produces novel text/images/code
Agentic AI   GenAI + tools + a loop + a goal
```

We're building agentic AI today. The agent is the **loop around the LLM**, not the LLM itself.

LLMs are stateless string-to-string machines. The state (tools, memory, goals) lives in *your* code.

Tokens, context, cost, latency

	what it is	rule of thumb
Token	~¾ of an English word	"Mechanical Keyboard" ≈ 3 tokens
Context window	tokens the model can see at once	128k for gpt-4o-mini today
Input cost	$ per million input tokens	$0.15/M for gpt-4o-mini
Output cost	$ per million output tokens	$0.60/M for gpt-4o-mini
Latency	time to first token + tokens/sec	~500ms TTFT + ~50 tok/s

A single agent step ≈ 500-2000 input tokens + 100-300 output. Plan accordingly.

Prompt engineering — the parts that actually matter

```
SYSTEM_PROMPT = (
    "You are a helpful assistant for a small product catalog. "
    "You have access to tools that let you list, search, count, add, "
    "and update products. Use them to answer the user's question. "
    "Prefer a single accurate tool call over multiple speculative ones."
)
```

Three things to get right:

1. **Role** — who the model is
2. **Constraints** — what it must/must not do
3. **Output shape** — JSON? prose? bullet list?

That's 80% of it. Skip "you are an expert" theatre.

Reusable prompt templates

```
from string import Template

PROMPT = Template(
    "Convert the user's question into a CatalogQuery JSON object.\n"
    "User: $user_question\n"
    "Schema: $schema"
)

prompt = PROMPT.substitute(
    user_question="electronics under 5000",
    schema=CatalogQuery.model_json_schema(),
)
```

Treat prompts as **strings in source control** — diff-able, reviewable, testable. Not docstrings, not Jupyter cells.

Structured outputs — the unlock

```
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[ ... ],
    response_format={"type": "json_object"},   # JSON mode
)
# Or, for strict schema validation:
response_format={"type": "json_schema",
                 "json_schema": {"schema": CatalogQuery.model_json_schema()}}
```

```
class CatalogQuery(BaseModel):
    category: Optional[str] = Field(default=None,
        description="Restrict to this category, or null for all.")
    max_price: Optional[float] = Field(default=None, ge=0)
    in_stock_only: bool = False
```

Pydantic + JSON mode = the LLM speaks your schema, not English you have to re-parse.

Validate the output, every time

```
raw = response.choices[0].message.content
try:
    query = CatalogQuery.model_validate_json(raw)
except ValidationError as exc:
    logger.warning("LLM returned invalid JSON: %s", raw)
    raise
```

Never trust the LLM's word that it produced your shape. **Always** parse through Pydantic. The same boundary discipline as Day 2 — the LLM is just another untrusted source.

Responsible AI — the minimum

- **PII**: don't send personal data to a third-party LLM without consent + a DPA in place
- **Hallucination**: assume the model can lie about numbers; **verify with a tool** before acting
- **Prompt injection**: any string from a user can attempt to override your system prompt — treat tool outputs the same way
- **Cost runaway**: cap `max_steps` on every agent
- **Audit**: log every prompt + tool call + result, with a request id

Not a course on responsible AI. But your boss will ask.

🧪 Lab 10 — NL Query → Catalog Filter

80 min · open `labs/lab-10-nl-query-filter.md`

You'll build:

- `CatalogQuery` Pydantic schema
- `parse_nl_query(prompt)` — one LLM call, JSON mode, Pydantic validation
- `apply_query(query, api)` — pure Python filter

End state: typing "electronics under 5000 in stock" returns the right rows.

Module 2

Tools as Python Functions + Agent Loop

~40 min · then 80 min lab

The big idea

> *In GenAI, a "**tool**" is just a Python function the LLM is allowed to call.*

That's it. No magic. No framework required.

You hand the LLM:

- A list of function **signatures** (as JSON schemas)
- The user's question

The LLM returns:

- "*Call* `function_X` *with arguments* `{ ... }`"

Your code:

- Looks up `function_X`, calls it, sends the result back
- Lets the LLM decide what to do with the observation

A tool is the Day-1 decorator, reimagined

```
@registry.tool(
    name="search_products",
    description="Find products whose name contains the given substring.",
    parameters={
        "type": "object",
        "properties": {"term": {"type": "string"}},
        "required": ["term"],
    },
)
def search_products(term: str) → list[dict]:
    return [p.model_dump() for p in self.api.list_products()
            if term.lower() in p.name.lower()]
```

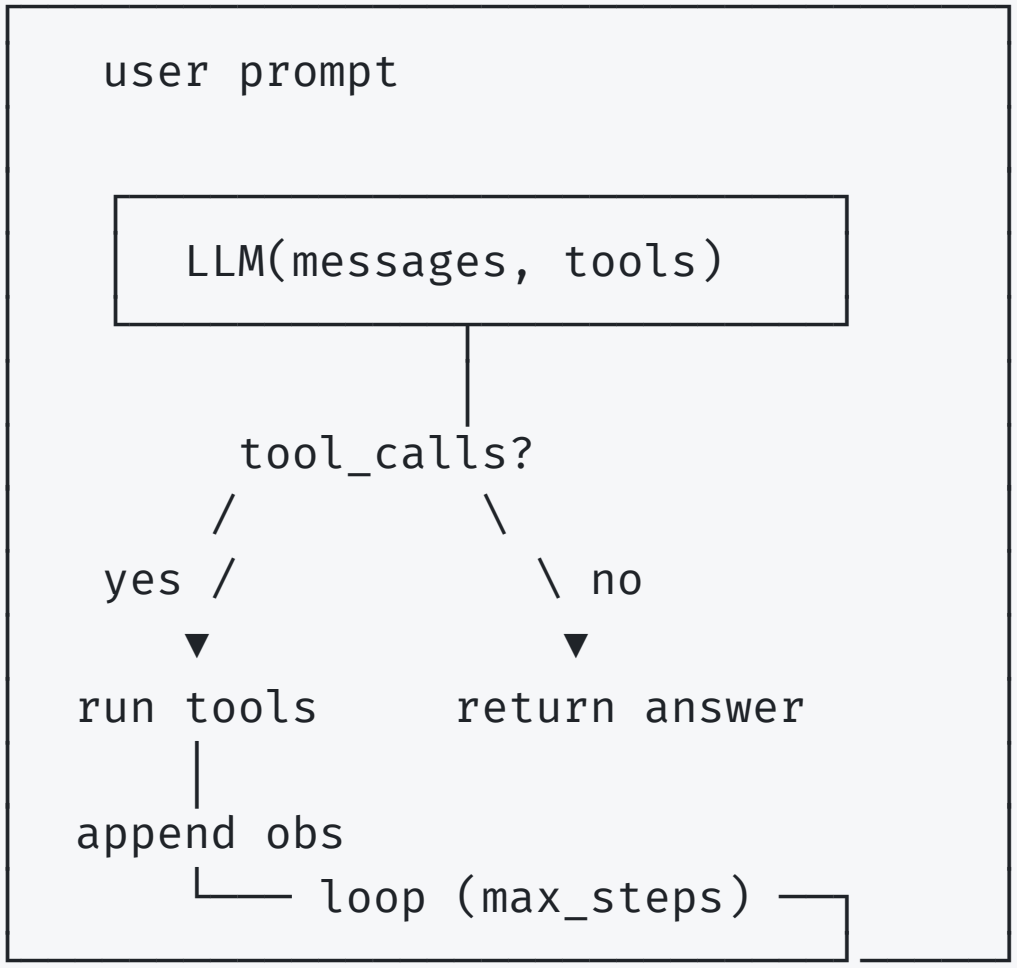
The `@tool` decorator stamps metadata on the function and adds it to a registry. Exactly the shape of Day 1's `@log_calls`.

Tool schema — what the LLM sees

```
{
  "type": "function",
  "function": {
    "name": "search_products",
    "description": "Find products whose name contains the given substring.",
    "parameters": {
      "type": "object",
      "properties": {"term": {"type": "string"}},
      "required": ["term"],
      "additionalProperties": false
    }
  }
}
```

A few words of `description=` are the **most important text in your codebase** for an agent. The LLM picks tools by reading them.

The agent loop



The agent loop, in code

```
def ask(self, user_prompt: str) → AgentResult:
    messages = [{"role": "system", "content": SYSTEM_PROMPT},
                {"role": "user",   "content": user_prompt}]
    log = []
    for step in range(1, self.max_steps + 1):
        resp = self.llm.chat.completions.create(
            model=self.model, messages=messages,
            tools=self.registry.openai_schemas())
        msg = resp.choices[0].message
        if not msg.tool_calls:
            return AgentResult(answer=msg.content, tool_calls=log, steps=step)
        for call in msg.tool_calls:
            result = self._invoke_tool(call.function.name, call.function.arguments)
            log.append(ToolCallRecord(call.function.name, … , result))
            messages.append({"role": "tool", "tool_call_id": call.id, …})
    raise AgentError("did not converge")
```


About 30 lines. That's the whole "agent".

Always cap `max_steps`

```
class CatalogAgent:
    def __init__(self, ... , max_steps: int = 5):  ...

    def ask(self, prompt):
        for step in range(1, self.max_steps + 1):
            ...
        raise AgentError(f"did not converge in {self.max_steps} steps")
```

Without a cap, a confused model loops forever. With a cap, you fail loudly. **Tomorrow's tests will assert this happens** — that's how you know the safety net works.

Memory & context, briefly

- **Short-term**: the `messages` list in the loop = working memory for one conversation
- **Long-term**: persist messages to a DB keyed by `session_id`, load on next `ask()`
- **Tools as memory**: a `remember(key, value)` tool that writes to a dict
- **Retrieval**: when the catalog grows past the context window, switch to RAG

We're not building memory today. But the pattern is the same: more Python around the LLM, not more LLM magic.

🧪 Lab 11 — Build the `CatalogAgent`

80 min · open `labs/lab-11-catalog-agent.md`

You'll build:

- `ToolSpec` + `ToolRegistry` + `@registry.tool( ... )` decorator
- `CatalogAgent.ask()` running plan/act/observe
- Four tools wrapping the Day-2 `APIClient`
- A demo: "what's our most expensive product?" → tool call → answer

Module 3 ★

Testing & Validating AI Tools and Outputs

the spine module — Day 3 patterns applied to AI

Why AI needs a different testing strategy

Traditional code:

```
same input → same output → assertEqual()
```

AI code:

```
same input → distribution of outputs → ?
```

You can't assert on prose. You can assert on:

- **Shape** (Pydantic schema)
- **Behaviour** (which tools were called, in what order)
- **Substrings** (the answer mentions the key fact)
- **Bounds** (≤ N tool calls, ≤ M tokens)

Stop testing prose. Start testing shape and behaviour.

Four classes of tests for an AI system

```
1.  Tool tests          — each tool is plain Python
2.  Schema tests        — LLM JSON validates against Pydantic
3.  Loop tests          — mock the LLM, verify the orchestration
4.  Golden evals        — file of cases that lock behaviour
```

Notice: **three of the four don't need an API key**. CI doesn't pay OpenAI.

1. Tool tests — deterministic Python

```python
class TestTools:
    def test_search_products_is_case_insensitive(self):
        agent = _make_agent()
        result = agent.registry.get("search_products").fn(term="KEYBOARD")
        assert result[0]["id"] == 2
```

Same shape as every other unit test you wrote on Day 3. The tools are *not magic*; they're functions. Test them as such.

2. Schema tests — Pydantic validation

```python
class TestCatalogQuerySchema:
    def test_rejects_negative_price(self):
        with pytest.raises(ValidationError):
            CatalogQuery(max_price=-5.0)

    def test_apply_query_filters_by_category_and_price(self):
        api = _fake_api(SAMPLE_PRODUCTS)
        q = CatalogQuery(category="Electronics", max_price=1000.0)
        assert {p["id"] for p in apply_query(q, api)} == {1}
```

Pure Pydantic + pure Python. The LLM is not involved.

3. Loop tests — mock the LLM (Day 3 deja vu)

```
def _llm_message(content=None, tool_calls=None):
    msg = MagicMock()
    msg.content = content
    msg.tool_calls = tool_calls or None
    return msg

class TestAgentLoop:
    def test_single_tool_call_then_answer(self):
        agent = _make_agent()
        agent.llm.chat.completions.create.side_effect = [
            _llm_response(_llm_message(
                tool_calls=[_tool_call("c1", "count_by_category")])),
            _llm_response(_llm_message(content="We have 3 Electronics.")),
        ]
        r = agent.ask("how many electronics?")
        assert [c.tool for c in r.tool_calls] == ["count_by_category"]
```

3b. Loop tests — runaway protection

```python
def test_max_steps_hit_raises(self):
    agent = _make_agent()
    agent.llm.chat.completions.create.return_value = _llm_response(
        _llm_message(tool_calls=[_tool_call("c1", "count_by_category")]))
    with pytest.raises(AgentError, match="did not converge"):
        agent.ask("loop forever")
```

The safety net you wrote in Lab 11 has a test. If a future refactor accidentally removes `max_steps`, CI catches it.

4. Golden evals — a file of cases

```
[
  { "id": "eval-01",
    "prompt": "How many products are in the Electronics category?",
    "expected_tool_calls": ["count_by_category"],
    "expected_answer_contains": ["Electronics"] },

  { "id": "eval-02",
    "prompt": "What's the most expensive product?",
    "expected_tool_calls": ["list_products"],
    "expected_answer_contains": ["Mechanical Keyboard"] }
]
```

Parametrize over the file. Add a row every time a real bug ships. The eval file becomes your **regression suite for behaviour**.

Parametrize over the golden file

```python
@pytest.mark.eval
class TestGoldenQueries:
    @pytest.mark.parametrize(
        "case", _golden_cases(),
        ids=[c["id"] for c in _golden_cases()],
    )
    def test_case_runs_expected_tools(self, case):
        agent = _make_agent()
        scripted = _build_scripted_responses(case)
        agent.llm.chat.completions.create.side_effect = scripted

        result = agent.ask(case["prompt"])

        assert [c.tool for c in result.tool_calls] == case["expected_tool_calls"]
        for needle in case["expected_answer_contains"]:
            assert needle in result.answer
```

What about testing against a real LLM?

Option A — never. Mock everything, ship.

Option B — `@pytest.mark.live_llm` opt-in tests, run nightly, never in CI.

Option C — record once with `vcrpy`, replay forever.

For Acuity: start with A. Move to C once you have a stable set of evals.

Live tests against a real LLM in CI = **cost runaway + flaky CI**.

AI-assisted test generation (responsibly)

```
"Here's my CatalogAgent class. Suggest 5 edge cases I'm probably not testing."
```

The LLM is great at:

- Listing edge cases you'd forget (negative numbers, empty strings, unicode)
- Drafting parametrize tables
- Pointing out untested branches

It's bad at:

- Knowing what *matters* in your domain
- Stable test data
- Maintenance

Use it for the boilerplate; keep the judgment.

Wire agent tests into the same CI

```
# .github/workflows/tests.yml — already in place from Day 3
- run: pytest --cov --html=report.html
```

No new workflow. The agent tests run alongside the model + client tests because they all live under `tests/`. **One green check** covers the whole stack.

```
✓ test (3.10)   1m 31s
✓ test (3.11)   1m 28s
✓ test (3.12)   1m 24s     53 tests including agent
```

🧪 Lab 12 — Test the Agent ⭐

80 min · open `labs/lab-12-test-the-agent.md`

You'll write:

1. Tool tests (deterministic Python)
2. Schema tests (Pydantic validation)
3. Loop tests with mocked LLM
4. Golden evals from `tests/evals/golden_queries.json`

End state: `pytest -q` green, no `OPENAI_API_KEY` needed, ~53 tests.

End of Day 4 ✅

Your `product-catalog/` repo:

- Python catalog + decorators + type hints (Day 1)
- FastAPI + Pydantic + `APIClient` + bulk-import (Days 1-2)
- pytest + mocks + parametrize + HTML reports + CI (Day 3)
- LLM-powered `CatalogAgent` with **its own test suite** (Day 4)

One project. Four days. Tested. Agentic. Done.

Where to next

- Replace the in-memory store with Postgres → integration tests catch the migration
- Swap OpenAI for Anthropic / Azure / a local model — only one file changes
- Add memory: persist `messages` per session_id
- Add retrieval: when the catalog grows, embed → vector search → top-k as context
- Take the patterns home: every one of them works on your real production code